Reflection on the Two Advanced Topics – Tetris Trials

Lars van de Corput & Gijs Bolwerk – Group 45

Git branching workflows

We chose to use Git branching workflows because we need to work together on the same project. Using git makes this easy for us, as we can use one workflow and commit and push things together. So, this really makes working together easier.

From researching, we learned how to push and commit and about repositories. We also learned that you could work on different branches, although we didn't take advantage of this except for the backup branches we used at the end, since merging was giving a bit of trouble. We mainly pushed on main, and that worked fine for us. Our merges were easy and not much of a problem. We should have used branches more though, since a giant merge in the end was difficult, caused some problems and thus took a long time and trial and error to resolve. So, using branches next time would be way more useful.

From the implementation, we learned how to use and bring it all together. The hardest part was getting NetBeans running with git. It was hard to set it all up using only NetBeans, so we ended up using a combination of committing via NetBeans and pushing and pulling via GitHub Desktop. Using GitHub Desktop in this way was helpful and thus we also learned how to use this. As I said in the previous paragraph, a failed merge due to a merge conflict left some of our files in an older state because we had trouble to resolve it, but we used backups in the branches to resolve it. Next time we should maybe also use smaller commits, as some were quite large and could thus give a lot of issues.

Lambda expressions

We chose to use lambda expressions since our game was using objectives, buffs and debuffs. Because each one works differently, using lambdas was a great way to work around this. Otherwise, there would have to be long if statements about every objective, or you would need a different class for every objective and debuff. By using lambdas, every objective and effect (buff and debuff) can have its own behaviour, acting like a mini method inside the big method. This way, adding new objectives or effects is also easy. The biggest reason was thus to reduce repetitive code.

From researching, we learned about functional interfaces. These are interfaces with only one method, and a lambda can use this in a single line. There are build in versions, such as Predicate. You can use method references, such as LockEvent::linesCleared (which we used). This way the code becomes easier to read and shorter, because we can refer to the method instead of using a lambda to call it. We also learned that methods could take or return lambdas. This way you can re-use a lot of previous made stuff. Using a Predicate to check conditions for true or false was also useful. For example, we used Predicate<LockEvent> to check if hold is used or not. Another important part is that lambdas can only use variables that aren't reassigned after initialization, so they are effectively final.

From the implementation, we learned how to use lambdas for an actual game. Every time the piece locks, the game makes a record called LockEvent with inside lines cleared, points gained and if hold or hard drop was used. This allows us to have all the information in one object, instead of using many different variables. You can then use lambdas and method references to check this information quickly and see if objectives have been passed. The interface Rule determines how an objectives progress changes after a LockEvent. By using re-usable rules, each objective could simply be a single line. For example, clear a double is Rules.countIf(e -> e.linesCleared() >= 2). This checks the number of LockEvents in which at least 2 lines were cleared. Buffs and debuffs work in the same way, but each effect has one lambda for start and end. We also used lambdas for movement and timers. For the writing to file, we used the file handling methods from Java to read and write the high scores, together with lists to store and sort them. The hard part was that lambdas do not work with changing variables, since a lambda can't reassign a local variable from another method. Because of that, the progress had to be stored inside the Objective class. There we used onLock() to pass the progress and the LockEvent to Rule, which return the progress. Objective stores this value, so that the progress is stored between events.